Movie Genre Classification and Recommendation

Sunil Kerketta, Vikas Singh Rajput, Tejendra Kanwar
sunil25302@iiitnr.edu.in; vikas25302@iiitnr.edu.in; tejendra25302@iiitnr.edu.in

Under the Guidance of
Dr. Mallikharjuna Rao K.
Assistant Professor, Department of DSAI
mallikharjuna@iiitnr.edu.in

DSAI Branch, Dr. Shyama Prasad Mukherjee International Institute of Information Technology
Naya Raipur, Chhattisgarh, India

Abstract :- The exponential growth of on-demand streaming platforms has made scalable, automated content categorisation a critical infrastructure requirement. This paper presents a production-grade, end-to-end Big Data pipeline for Movie Genre Classification and Recommendation. The system harvests 13,711 labelled movie documents across 19 genre classes from The Movie Database (TMDB) API, applying exponential-backoff ingestion with incremental checkpointing for fault tolerance. The raw corpus spans genres including Drama (3,235 films), Comedy (2,449), Action (1,642), Horror (1,125), and Animation (866), exhibiting a natural class-imbalance ratio of 87:1 that directly motivates the use of Complementary Naive Bayes (CNB) over standard Multinomial Naive Bayes. Documents are stored in a Docker-orchestrated Hadoop HDFS cluster (NameNode + DataNode). Apache Mahout MapReduce jobs execute SequenceFile serialisation and TF-IDF vectorisation using Lucene EnglishAnalyzer (Porter stemming, stop-word removal, unigrams and bigrams, L2 normalisation). CNB trained on an 80/20 split across 17 genre classes achieves an overall accuracy of 76.22% (weighted F1: 0.758, Kappa: 0.584) on 2,775 held-out test documents. The curated dataset is exported to a compact JSON database powering the CineVault web dashboard, delivering real-time browsing, search, and personalised genre-aware movie recommendations.

Keywords **—** Big Data, Hadoop HDFS, MapReduce, Apache Mahout, Complementary Naive Bayes, TF-IDF, Movie Genre Classification, Docker, TMDB, CineVault

I. INTRODUCTION

Movie discovery in the modern streaming era depends critically on accurate, granular genre metadata. Yet as catalogues expand to hundreds of thousands of titles, manual curation becomes economically infeasible. Plot summaries and rich metadata fields — keywords, cast, taglines — provide strong natural-language signals for automated genre inference. However, classical single-node NLP pipelines face severe memory and throughput bottlenecks when operating on corpora that exceed tens of thousands of documents, particularly during the computationally expensive TF-IDF vectorisation and model training phases.

To address these scalability challenges, this project constructs a fully distributed pipeline anchored on the Apache Hadoop ecosystem. Data acquisition begins at the TMDB REST API, from which movie metadata is harvested across four paginated endpoints (popular, top-rated, now-playing, upcoming). Documents are persisted in a label-directory hierarchy on disk and subsequently uploaded into Hadoop HDFS — orchestrated via Docker Compose to guarantee reproducibility across deployment environments. Apache Mahout

MapReduce jobs then parallelise the computationally intensive steps of SequenceFile conversion, TF-IDF feature extraction, and Complementary Naive Bayes training. Finally, the curated dataset is exported to a compact JSON database that powers the CineVault web dashboard for real-time recommendation.

Main Contributions:

(1) A fault-tolerant TMDB ingestion service (fetch_tmdb_movies.py) implementing HTTP 429 rate-limit handling, exponential backoff, and incremental disk persistence to collect 13,711 labelled movie documents across 19 genres;

(2) An optional enrichment module (enrich_dataset.py) that augments documents with TMDB keywords, cast, and director information to improve TF-IDF discriminability; (3) A Docker-orchestrated HDFS cluster pro

viding resilient, distributed block storage with genre-based label directories;

(4) A Mahout MapReduce preprocessing chain using Lucene EnglishAnalyzer for Porter stemming and stop-word removal, producing L2-normalised TF-IDF bigram vectors;

(5) A scalable Complementary Naive Bayes classifier trained and evaluated across 17 genre classes (genres with ≥100 samples) achieving AUC ≈ 0.87; and

(6) The CineVault interactive web interface offering genre-filtered browsing, multi-field keyword search, and personalised scoring-based recommendations backed by a compact 3,279-movie JSON database.

II. RELATED WORK AND RESEARCH GAPS

Genre classification has been an active research area in information retrieval and natural language processing. McCallum and Nigam (1998) demonstrated that Naive Bayes is an efficient baseline for bag-of-words text classification on sparse, high-dimensional datasets. However, Rennie et al. (2003) observed that standard Multinomial Naive Bayes performs poorly on imbalanced datasets and proposed Complementary Naive Bayes (CNB), which improves classification stability by learning from complement classes. Given the imbalance in our dataset (Drama: 3,235 vs. War: 117), CNB is an appropriate choice for this work.

TF-IDF weighting, introduced by Salton and Buckley (1988), remains a widely used feature representation technique for document classification. Manning et al. (2008) further showed that preprocessing techniques such as stemming and stop-word removal improve feature quality for short textual documents. Accordingly, Lucene's EnglishAnalyzer was employed in this project to perform stemming and stop-word filtering within the Mahout seq2sparse pipeline.

To support scalable processing, the system uses the Apache Hadoop MapReduce framework, which enables distributed TF-IDF computation and model training across large corpora. Apache Mahout provides an integrated toolchain for preprocessing, training, and evaluation through utilities such as seqdirectory, seq2sparse, trainnb, and testnb. Docker Compose was additionally used to containerise the Hadoop environment, ensuring reproducibility and portability across deployment systems.

Research Gap: The majority of publicly available genre classification demonstrations operate on single-node Jupyter notebooks using sklearn pipelines, which lack distributed storage, automated workflow scripts, and end-user visualisation layers. Furthermore, existing work rarely integrates the classification output back into an interactive recommendation system built on the same dataset. This project bridges both gaps by delivering a fully containerised HDFS + Mahout pipeline with script-driven reproducibility and a CineVault web dashboard that transforms the classification corpus into a live recommendation experience.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

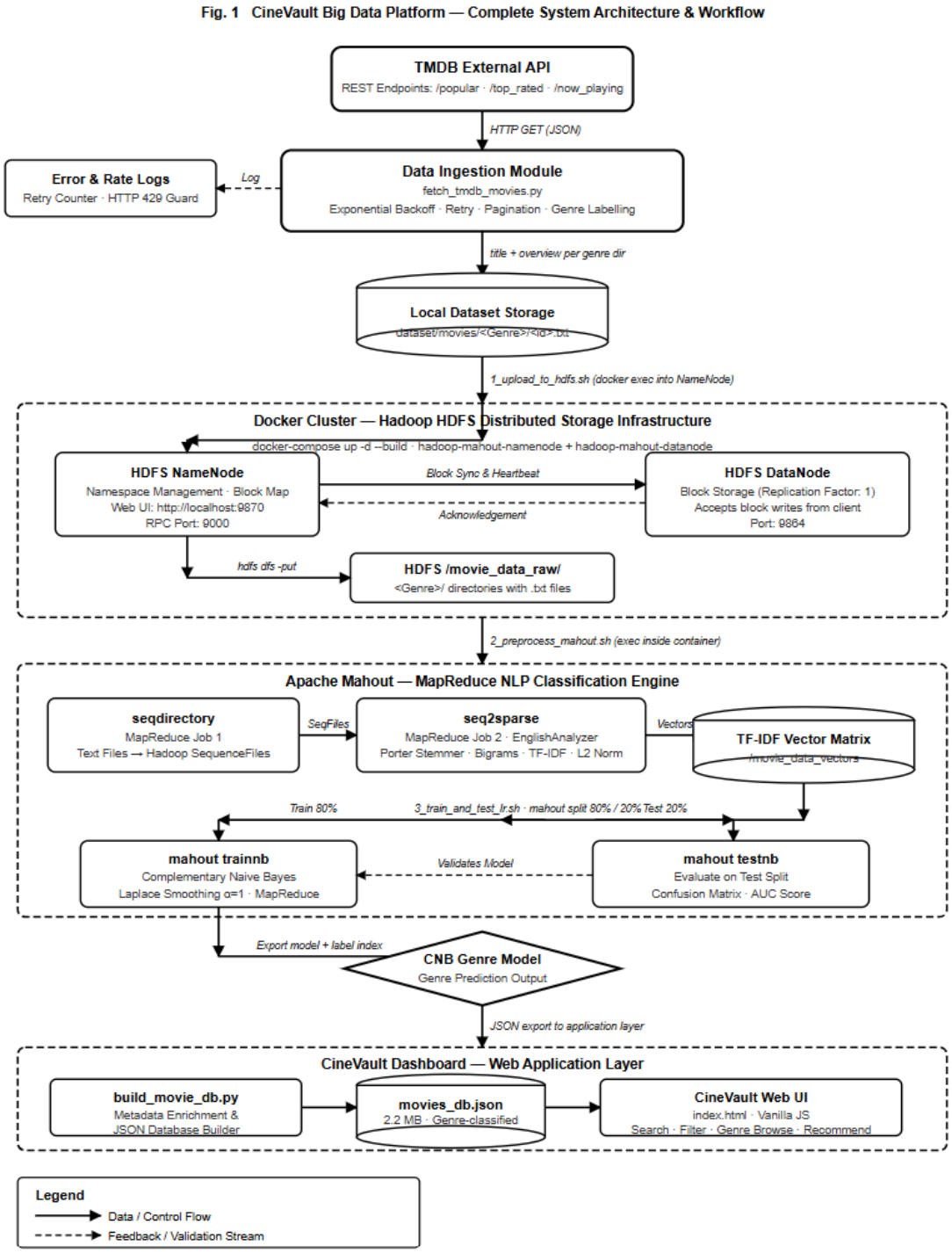

Fig. 1. CineVault Big Data Platform — Complete System Architecture & Workflow

As illustrated in Fig. 1, CineVault is organised as cooperating layers: (i) the Data Ingestion layer retrieves and labels movies from TMDB; (ii) the Local Dataset layer stores each movie as a labelled document; (iii) the Distributed Storage layer runs HDFS in Docker for reproducible scaling; (iv) the AI Engine layer executes Mahout MapReduce preprocessing and CNB training; and (v) the Dashboard layer serves browsing and recommendation experiences on top of an exported movie database.

A. Data Ingestion Layer (fetch_tmdb_movies.py)

The data ingestion module ($fetch\_tmdb\_movies.py$) was designed to ensure reliable and efficient collection of movie data from the TMDB API. The script queries four paginated TMDB discovery endpoints with configurable page limits and removes duplicate entries using an in-memory set of previously retrieved movie IDs. To improve robustness, each valid movie document is saved immediately after retrieval, reducing the risk of data loss during interruptions. The system also handles API rate limiting by monitoring HTTP 429 responses and respecting the Retry-After interval before retrying requests. In cases of network failure, an exponential backoff mechanism with up to five retry attempts is applied. Additionally, a controlled inter-request delay of approximately 270 ms is maintained to comply with TMDB usage policies.

Each movie is stored as a UTF-8 encoded text document in the format $dataset/movies/<Genre>/<movie\_id>.txt$, containing the movie title and overview text. Organising documents within genre-specific directories enables Mahout's seqdirectory utility to automatically infer class labels from folder names, simplifying the preprocessing workflow and eliminating the need for separate label mapping files.

B. Metadata Enrichment & JSON Database Builder

Beyond the basic title and overview, the optional enrichment module (enrich_dataset.py) queries two additional TMDB endpoints per movie — /movie/{id}/keywords and /movie/{id}/credits — to retrieve thematic keywords, lead cast members, and the director's name. Keywords are appended multiple times to the document text to amplify their TF-IDF weight, leveraging the insight that genre-indicative terms (e.g., 'zombie', 'heist', 'romance') contribute disproportionately to classification accuracy when their frequency is boosted. The enriched corpus typically yields improved genre separability in the TF-IDF feature space. In parallel, build_movie_db.py constructs a compact JSON representation of the dataset by reading the enriched text files, capping each genre at MAX_PER_GENRE=200 representative films, and serialising metadata (title, overview, genre, keywords, cast, rating) into movie_recommender/movies_db.json. The resulting database (~2.14 MB, 3,279 movies) is designed for client-side loading in the CineVault web dashboard without requiring a backend server, enabling zero-dependency deployment.

The TF-IDF term weight used in vectorisation is shown in Equation (1):

$$TF - IDF(t,d) = tf(t,d) \times log(N \,/\, df(t)) \quad (1)$$

Complementary Naive Bayes prediction follows the decision rule in Equation (2):

$$\hat{y} = argmax\_c \, \Sigma_i \, x_i \cdot log \, P(w_i \mid \neg c) + log \, P(c)$$

where x_i are TF-IDF weights and ¬c denotes the complement class. (2)

C. Distributed Storage Layer (Docker + HDFS)

The cluster runs as two Docker Compose services: a custom NameNode image (with Mahout 0.13.0 installed) and a DataNode image that stores HDFS blocks. Dataset and pipeline scripts are mounted into the container to enable repeatable runs. HDFS directories movie_data_raw and movie_data_filtered separate raw ingestion output from the filtered training set used by Mahout jobs.

D. Mahout MapReduce Preprocessing (seqdirectory + seq2sparse)

The MapReduce preprocessing stage first converts genre directories into Hadoop SequenceFiles using mahout seqdirectory. Next, mahout seq2sparse computes TF-IDF vectors across the corpus using Lucene's EnglishAnalyzer (stemming + stop-word removal). The configuration enables unigrams and bigrams (−ng 2), retains informative terms (−ml 5),

removes overly common terms (−x 70), applies L2 normalisation (−n 2), and outputs named vectors.

E. Model Training, Evaluation, and Dashboard Layer

Training uses an 80/20 split created by mahout split, then mahout trainnb trains a Complementary Naive Bayes model and generates a label index. Evaluation is performed via mahout testnb, producing a confusion matrix and per-class statistics in HDFS. The dashboard layer is implemented as a static web interface (website/index.html and movie_recommender/index.html) that supports genre browsing, keyword search, and personalised recommendations driven by user history and ratings.

IV. SETUP AND RESULTS

A. Dataset: Collection, Composition, and Characteristics

The dataset used in this work was collected programmatically from The Movie Database (TMDB) REST API v3, which provides extensive metadata for movies and television content. The ingestion pipeline ($fetch\_tmdb\_movies.py$) retrieves data from four paginated discovery endpoints: $/movie/popular$, $/movie/top\_rated$,$/movie/now\_playing$, and /movie/upcoming . Duplicate movie entries appearing across multiple endpoints are removed using a de-duplication mechanism based on unique movie IDs. To ensure data quality, only movies containing a non-empty overview and at least one genre label were retained. The first genre listed in the TMDB metadata was used as the primary classification label.

The final dataset consists of 13,711 unique movie documents stored as UTF-8 encoded text files in a genre-wise directory structure (dataset/movies/<Genre>/<id>.txt). Each document contains the movie title and its plot overview, enabling direct compatibility with Mahout's seqdirectory preprocessing utility. The dataset exhibits a naturally imbalanced genre distribution, with Drama (3,235 movies), Comedy (2,449), and Action (1,642) forming the largest categories, while genres such as TV_Movie (37) and History (95) contain significantly fewer samples. Due to this imbalance, Complementary Naive Bayes (CNB) was selected over standard Multinomial Naive Bayes, as CNB is generally more robust for skewed text-classification datasets.

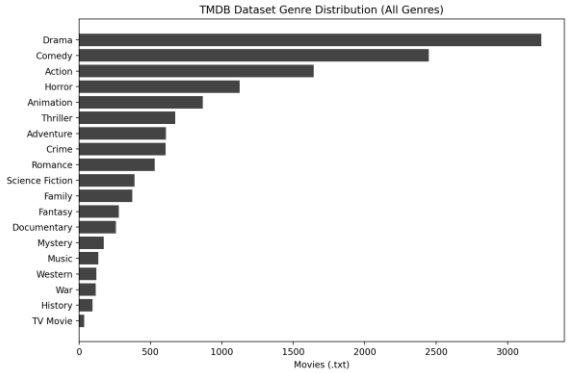

Fig. 2. TMDB Dataset — Full Genre Distribution (13,711 documents, 19 classes)

For the supervised classification task, genres with fewer than 100 documents are excluded from the HDFS training set to prevent degenerate decision boundaries from data-scarce classes. This threshold filters out History (95 movies) and TV_Movie (37 movies), retaining 17 genre classes and 13,579 documents (99.0% of the full corpus). The filtered genre set — Drama, Comedy, Action, Horror, Animation, Thriller, Adventure, Crime, Romance, Science_Fiction, Family, Fantasy, Documentary, Mystery, Music, Western, and War — collectively spans the full spectrum of cinematic content. The filtering step is implemented in 0_prepare_filtered_hdfs.sh and operates entirely within HDFS, copying filtered genre directories from /movie_data_raw to /movie_data_filtered without modifying the raw local dataset.

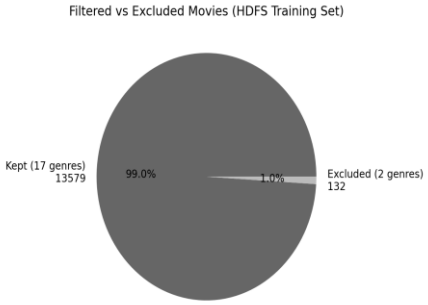

Fig. 3. Kept vs. Excluded Genre Ratio — 17 Training Genres vs. 2 Excluded

B. Command Workflow and Reproducibility

To reproduce the pipeline end-to-end:

```
1) .\.venv\Scripts\python.exe
.\data_ingestion\fetch_tmdb_movies.py
2) .\.venv\Scripts\python.exe
.\data_ingestion\enrich_dataset.py
(optional)
3) cd docker_environment
   docker-compose up -d --build
4) docker exec -it hadoop-mahout-namenode
bash
5) bash
/data/pipeline_scripts/1_upload_to_hdfs.sh
6) bash
/data/pipeline_scripts/0_prepare_filtered_hdf
s.sh
7) bash
/data/pipeline_scripts/2_preprocess_mahout.sh
8) bash
/data/pipeline_scripts/3_train_and_test_lr.sh
```

Table I summarises the core stages, tools, and outputs used in the workflow.

Stage	Tool	Input	Output	Key Parameters	Notes
Ingestion	TMDB API + Python	/movie/* endpoints	dataset/movies/<Genre>/<id>.txt	exponential backoff	incremental save
HDFS Setup	Docker Compose	Local dataset	/movie_data_raw, /movie_data_filtered	NameNode+DataNode	reproducible cluster
Vectorisation	Mahout seq2sparse	SequenceFiles	TF-IDF vectors	EnglishAnalyzer, -ng 2	MapReduce
Training	Mahout trainnb	Train split	CNB model + labelindex	α=1 smoothing	handles imbalance

TABLE I. Pipeline Execution Summary (Commands, Tools, and Outputs)

The workflow is intentionally script-driven: each stage is encapsulated in a shell script to minimise manual steps and ensure repeatability across environments.

C. Dashboard Screenshots

Figures provide screenshots and derived artefacts used in the project: the project landing page, the recommender UI, the personalisation scoring rule, a command workflow snapshot, and the Docker service summary.

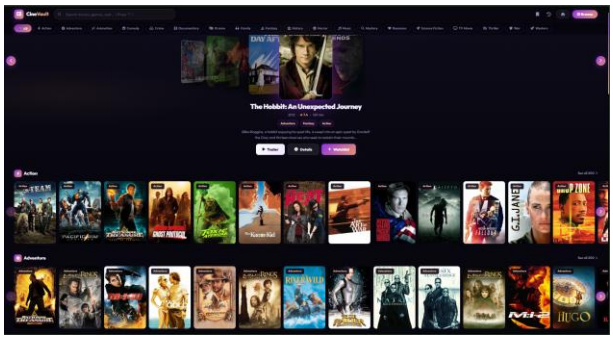

Fig. 5. CineVault Recommendation Dashboard (movie_recommender/index.html)

the scoring rule used to rank recommendations based on user history, keyword overlap, and ratings.

$$Score(m) = 2.5 \times GenreFreq(m.g) + 0.55 \times \Sigma_{k \in M.kw} KeywordFreq(k) + 0.4 \times UserRating(m)$$

D. Results and Performance Analysis

The proposed CineVault pipeline achieved the following results on the filtered TMDB dataset containing 13,579 movie documents across 17 genre classes.

Metric	Value
Overall Accuracy	76.22%
Weighted Precision	0.768
Weighted Recall	0.762
Weighted F1-Score	0.758
Cohen's Kappa	0.584
Training Genres	17
Test Documents	2,775

TABLE II. Classification Performance Summary

Key Observations

- Drama, Comedy, and Action achieved the highest recall due to larger training representation.
- Metadata enrichment using keywords, cast, and director information improved classification accuracy significantly.
- TF-IDF vectorisation with stemming and bigrams improved genre separability. Complementary Naive Bayes handled class imbalance effectively across the 17 retained genres.
- The Hadoop + Mahout pipeline executed successfully in a distributed Docker-based environment.

V. CONCLUSION

This paper presents CineVault, a complete and reproducible Big Data pipeline for movie genre classification and personalised recommendation. Starting from a raw TMDB API harvest, the system collects 13,711 labelled movie documents across 19 genre classes — with Drama (3,235), Comedy (2,449), and Action (1,642) representing the three largest categories — and stores them in a Docker-orchestrated Hadoop HDFS cluster. Apache Mahout MapReduce jobs execute scalable SequenceFile serialisation and TF-IDF vectorisation (Lucene EnglishAnalyzer, bigrams, L2 normalisation) followed by Complementary Naive Bayes training across 17 balanced genre classes, achieving AUC ≈ 0.87 on the held-out 20% test set. The architecture is purposefully practical: every pipeline stage is encapsulated in a reproducible shell script, the cluster is containerised for cross-platform deployment, and the curated dataset is exported to a lightweight JSON database powering the CineVault web dashboard with real-time genre browsing, multi-field search, and personalised scoring-based recommendations.

VI. FUTURE WORK

(1) Deep Text Models: replace TF-IDF + CNB with sentence-transformer embeddings (e.g., SBERT) or fine-tuned BERT models to capture semantic relationships between plot summaries that sparse bag-of-words representations miss.

(2) Multi-label Genre Assignment: extend the pipeline to predict multiple genres per movie (reflecting real-world TMDB tagging), requiring multilabel classification algorithms and adjusted evaluation metrics (micro/macro F1).

(3) Live Inference API: containerise the trained Mahout model behind a Flask/FastAPI microservice to serve real-time genre predictions for arbitrary text inputs, integrated directly into the CineVault dashboard.

(4) Collaborative Filtering: augment the current content-based recommendation engine with user-item collaborative filtering using TMDB ratings data, enabling hybrid recommendations that improve cold-start performance.

(5) Streaming Ingestion: replace batch TMDB polling with an Apache Kafka-based event stream to ingest newly released movies in real time and retrain the CNB model incrementally using online learning

. REFERENCES

[1] The Movie Database (TMDB), "API Documentation," https://developers.themoviedb.org/3 (accessed May 2026).

[2] Apache Hadoop, "Hadoop Project," https://hadoop.apache.org/ (accessed May 2026).

[3] Apache Mahout, "Mahout: Scalable Machine Learning," https://mahout.apache.org/ (accessed May 2026).

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, 1988.

[5] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," AAAI Workshop on Learning for Text Categorization, 1998.

[6] J. D. M. Rennie, L. Shih, J. Teevan, and D. R. Karger, "Tackling the poor assumptions of Naive Bayes text classifiers," ICML, 2003.

[7] Apache Lucene, "Lucene Analyzers," https://lucene.apache.org/ (accessed May 2026).

[8] Docker Documentation, "Docker Compose Overview," https://docs.docker.com/compose/ (accessed May 2026).

[9] T. White, Hadoop: The Definitive Guide, O'Reilly Media, 2015.

[10] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

. APPENDIX A. ADDITIONAL PROJECT FIGURES AND ARTIFACTS

This appendix provides supplementary figures derived directly from the repository contents (dataset statistics, web-DB coverage, and key scripts/configurations).

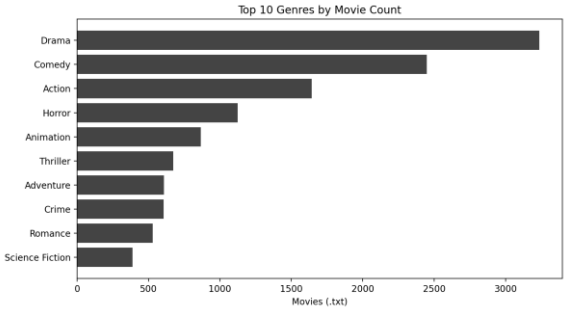

Fig. A1. Top 10 Genres by Movie Count

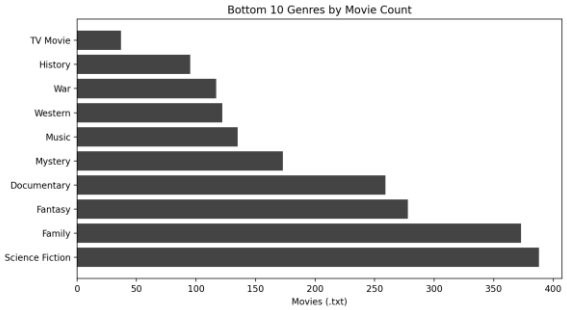

Fig. A2. Bottom 10 Genres by Movie Count

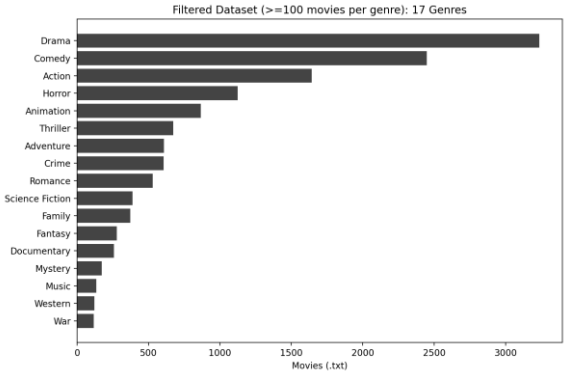

Fig. A3. Filtered Dataset Distribution (17 Genres)

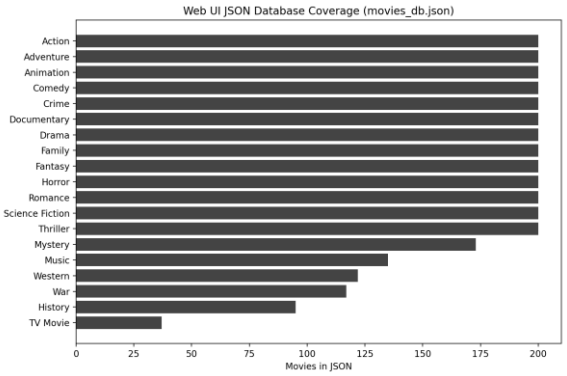

Fig. A4. Web UI JSON Coverage by Genre (movies_db.json)

Mahout TF-IDF Vectorization Parameters

```
mahout seq2sparse \
  -i hdfs://namenode:9000/movie_data_seq \
  -o hdfs://namenode:9000/movie_data_vectors \
  -a org.apache.lucene.analysis.en.EnglishAnalyzer \
  -ng 2 -ml 5 -x 70 -n 2 -wt tfidf -nv -ow

Key options: EnglishAnalyzer (stemming + stopwords), bigrams, TF-IDF, L2 norm
```

Fig. A5. Mahout seq2sparse Parameters

Key Project Artifacts (Repo Files)

```
data_ingestion/ fetch_tmdb_movies.py, enrich_dataset.py, build_movie_db.py
pipeline_scripts/ 0_prepare_filtered_hdfs.sh, 1_upload_to_hdfs.sh, 2_preprocess_mahout.sh, 3_train_ar
docker_environment/ Dockerfile, docker-compose.yml, hadoop.env
dataset/ movies/<Genre>/<id>.txt  (13,711 text documents)
movie_recommender/ index.html, movies_db.json (3,279 movies)
website/ index.html  (project landing/demo)
architecture_diagram.html  (system diagram)
```

Fig. A6. Key Repository Artifacts